STM32 USB 虚拟串口原理与程序设计

基于 USB CDC 的通信机制、软件结构与流程设计

周贤中

集成电路学院

广东工业大学

2026 年 4 月 20 日

目录

1 背景与定位

2 USB 通信基础

3 CDC 虚拟串口原理

4 STM32 中的 USB CDC

5 程序设计要点与流程图

目录

1 背景与定位

2 USB 通信基础

3 CDC 虚拟串口原理

4 STM32 中的 USB CDC

5 程序设计要点与流程图

为什么要学习 USB 虚拟串口

- 在嵌入式开发中，虚拟串口常用于**调试输出**、**运行日志**、**命令交互**和**上位机参数配置**。
- 相比传统 UART，USB 直接连接电脑更方便，不必额外准备 USB 转串口模块。
- 对教学实验而言，USB 虚拟串口能同时覆盖**枚举识别**、**双向通信**、**字符串处理**和**软件分层**等知识点。
- 后续实验通常都会建立在这套理论之上：先让电脑识别出 COM 口，再完成命令收发与业务逻辑。

从传统 UART 串口说起

UART 的核心特点

- 通过 TX/RX 两根信号线进行异步通信
- 依赖波特率、数据位、校验位、停止位等串口参数
- 常见配置写作 115200 8N1
- 其中 115200 表示波特率，8 表示 8 位数据位，N 表示无校验，1 表示 1 位停止位
- 数据按**位时序**逐位发送

补充说明

UART 是**异步通信**，没有单独的时钟线，所以通信双方必须事先约定这些参数。

8N1 的最小帧结构

典型一帧可理解为：

1 起始位 + 8 数据位 + 0 校验位 + 1 停止位

因而发送 1 字节有效数据，线路上通常需要传输 10 bit。

UART 与 USB 虚拟串口的本质区别

对比项	传统 UART 串口	USB 虚拟串口
物理基础	串口外设 + TX/RX 引脚	USB 外设 + D+/D- 差分线
通信单位	串行位流与帧格式	USB 数据包
建链方式	直接连线即可通信	先枚举，再加载驱动，最后才能通信
电脑侧表现	真正的串口硬件或 USB 转串口芯片映射而来	USB CDC 驱动在操作系统中创建出的 COM 接口
上层使用方式	串口助手打开端口收发文本	串口助手同样打开端口，但底层已变成 USB

结论：“虚拟串口”是**接口层面的相似**，不是**底层协议完全相同**。

目录

1 背景与定位

2 USB 通信基础

3 CDC 虚拟串口原理

4 STM32 中的 USB CDC

5 程序设计要点与流程图

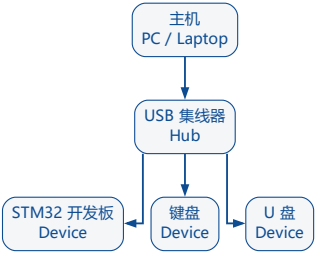
USB 的基本思想

- USB 是 **Universal Serial Bus**，即通用串行总线。
- 它不是简单的一根数据线，而是一整套**总线协议 + 供电机制 + 枚举机制 + 类规范**。
- USB 系统强调**即插即用**：设备接入后，主机会主动识别其类型并加载相应驱动。
- 对本课程的 STM32 场景来说，开发板扮演 **Device**，电脑扮演 **Host**。

USB 中的 Host 与 Device

USB 的主从关系

- **Host**（主机）负责供电、枚举、调度和发起通信
- **Device**（设备）负责响应主机请求，按协议提供功能
- **Hub**（集线器）用于扩展主机可连接的 USB 设备数量
- 总线访问权由主机掌握，设备不能像 UART 一样主动“抢线”



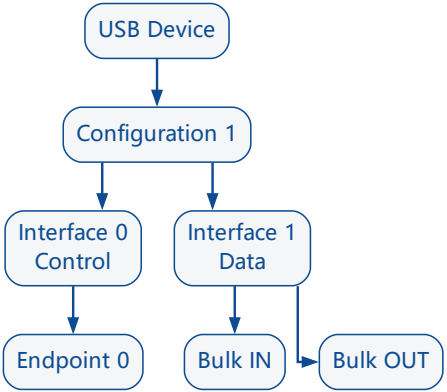
图中展示的是“一台主机通过集线器连接多个设备”的典型拓扑。若只连接一块 STM32 开发板，也可以不经过外置 Hub，直接与主机相连。

USB 设备的逻辑结构

- **Device**：整个 USB 设备。
- **Configuration**：设备的一组工作配置。
- **Interface**：某一类功能接口，例如控制接口、数据接口。
- **Endpoint**：真正承载数据传输的端点。

理解重点

主机不是直接面对“某个函数”，而是通过配置、接口和端点来理解设备能力。



Descriptor：主机为什么能识别设备

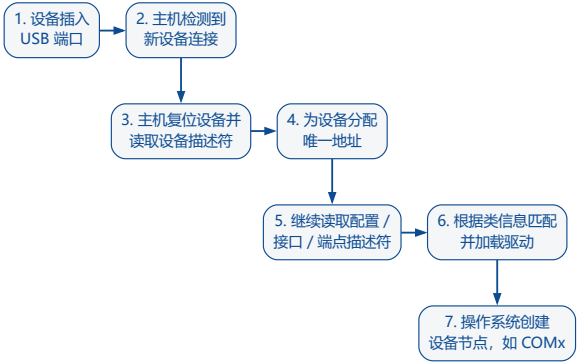
- USB 设备具备**自描述**能力，主机会主动读取各种描述符来理解设备。
- 常见描述符包括：
  - **设备描述符**：厂商 ID、产品 ID、USB 版本等
  - **配置描述符**：设备有哪些配置
  - **接口描述符**：每个接口属于什么类
  - **端点描述符**：每个端点的方向、类型和大小
  - **字符串描述符**：产品名、厂商名等可读文本
- 主机之所以知道“这是一个 CDC 设备”，本质上就是从这些描述符中读出来的。

USB 的四种传输类型

传输类型	特点	常见用途
Control	所有 USB 设备必须支持，主要用于枚举、配置和控制命令	读取描述符、设置地址、标准请求
Bulk	强调数据正确性，允许重传，但不保证固定实时性	虚拟串口、U 盘、大块数据传输
Interrupt	适合小数据量、周期性查询，延迟通常较小	鼠标、键盘、状态上报
Isochronous	强调时间连续性，不强调重传可靠性	音频、视频流

CDC 虚拟串口通常主要依赖 `Control` 和 `Bulk`。

USB 枚举过程


1. 设备插入 USB 端口
2. 主机检测到新设备连接
3. 主机复位设备并读取设备描述符
4. 为设备分配唯一地址
5. 继续读取配置 / 接口 / 端点描述符
6. 根据类信息匹配并加载驱动
7. 操作系统创建设备节点，如 COMx

目录

1 背景与定位

2 USB 通信基础

3 CDC 虚拟串口原理

4 STM32 中的 USB CDC

5 程序设计要点与流程图

USB 设备类与 CDC 的定位

- 为了让主机快速识别设备用途，USB 规范定义了不同的**设备类**。
- 常见类包括：HID、MSC、CDC、Audio、Video。
- **CDC** 即 **Communication Device Class**，原本用于通信类设备。
- 在嵌入式开发中，CDC 最常见的应用就是实现**虚拟串口**。

为什么 CDC 能成为“虚拟串口”

- 操作系统提供了与 CDC 兼容的驱动模型。
- 驱动会把 USB CDC 设备**抽象**成一个串口设备接口。
- 因此应用程序可以像访问普通串口一样打开 `COMx` 并收发数据。
- 也就是说：**上层像串口，底层是 USB 包传输。**

常见术语

- `VCP`
  Virtual COM Port
- `CDC ACM`
  Abstract Control Model

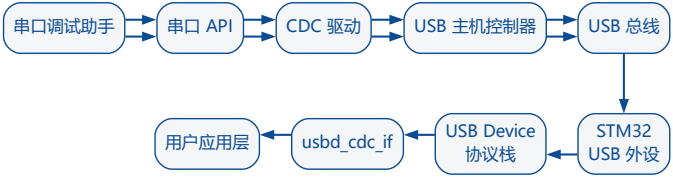
电脑上的 COM 口到底从哪里来

容易混淆的地方

当 STM32 作为 USB CDC 设备连接到电脑时，
电脑上出现的 COM 口并不是“主板多出了一路真实 UART”，
而是**操作系统中的驱动层创建出的软件接口**。

- 串口调试助手看到的是 COMx
- 驱动层处理的是 CDC 设备读写请求
- 主机控制器和 USB 总线传输的是 USB 数据包

USB 虚拟串口的数据收发路径



理解重点

串口助手虽然看起来在“直接和串口通信”，实际上中间经过了操作系统驱动和 USB 协议栈。

虚拟串口与传统 UART：相同点与不同点

相同点

- 上位机都能打开一个串口接口
- 都可以发送文本命令并接收文本响应
- 都可用于日志输出和调试终端

不同点

- UART 靠位时序，USB 靠数据包和协议栈
- UART 不需要枚举，USB 必须先枚举
- UART 常是点对点，USB 由主机统一调度
- UART 真正依赖波特率，CDC 通常不依赖串口助手里设置的波特率

为什么串口助手还要设置 115200 8N1

- 很多上位机软件是按“串口工具”的统一界面设计的，因此仍然要求填写波特率、数据位、校验位和停止位。
- 对 USB CDC 虚拟串口而言，这些参数通常**不会像传统 UART 那样真正决定底层 USB 传输速率**。
- 之所以仍常写成 `115200 8N1`，主要是为了：
  - 保持软件界面一致
  - 避免工具因参数为空而无法打开端口
  - 让实验文档书写格式统一

结论:在 USB CDC 场景下，`115200 8N1` 更多是**接口兼容信息**，而不是 USB 总线速率配置。

目录

1 背景与定位

2 USB 通信基础

3 CDC 虚拟串口原理

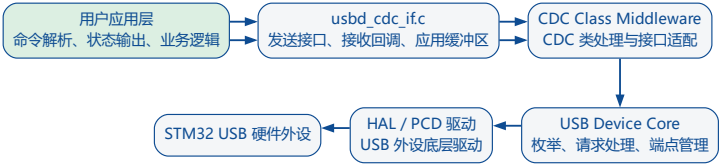
4 STM32 中的 USB CDC

5 程序设计要点与流程图

STM32 为什么能够实现 USB 虚拟串口

- 芯片必须具备 **USB Device** 外设，能够处理 USB 全速设备通信。
- 硬件上要有 `D+/D-` 引脚及相关连线条件。
- 软件上需要 USB Device 协议栈和 CDC 类中间件。
- 在 STM32CubeMX 中启用 `USB Device CDC` 后，系统会生成一套基础框架。
- 用户程序则在这套框架上补充发送、接收和命令处理逻辑。

STM32 USB CDC 的软件分层



为什么 USB 时钟必须精确到 48 MHz

- USB Full Speed 外设对时钟精度要求很高，STM32 的 USB 模块通常需要**准确的 48 MHz 时钟**。
- 如果时钟来源或分频设置错误，最直接的后果就是：
  - 电脑无法识别设备
  - 能识别但连接不稳定
  - 数据收发异常或偶发失败
- 因此，USB CDC 的问题排查往往首先要检查时钟树，而不是一上来就怀疑字符串处理代码。

工程结论

“电脑看不到 COM 口”时，**48 MHz** 配置错误通常是首要检查项之一。

STM32 USB CDC 的发送机制

- 用户层通常通过 `CDC_Transmit_FS()` 或其封装函数发送数据。
- 发送请求不会保证每次都立即成功，因为底层端点可能仍在发送上一包数据。
- 当接口返回 `USBD_BUSY` 时，说明当前发送资源尚未空闲。
- 因而一个稳定的发送函数一般要考虑：
  - 是否需要短时间重试
  - 是否设置超时，避免长时间卡死
  - 是否在应用层实现发送队列

设计启示

USB CDC 的发送不是“写一行函数就万事大吉”，而是要把**忙状态**当作正常情况来处理。

STM32 USB CDC 的接收机制

- 主机发送 USB 数据包后，STM32 的 USB 外设和协议栈会接收数据并回调到 CDC 接口层。
- 在典型工程中，`CDC_Receive_FS()` 负责：
  - 读取 `Buf` 与 `Len`
  - 将数据复制到应用层缓冲区
  - 记录接收长度
  - 置位“收到新数据”的标志
  - 重新开启下一次接收
- 真正的命令解析、字符串比较和业务处理，通常放在主循环或 RTOS 任务中完成。

为什么不建议在接收回调里做复杂业务

- 回调函数应尽量短，优先保证收包链路持续可用。
- 如果在回调里直接做复杂字符串处理、格式化输出或耗时逻辑，会导致：
  - 回调执行时间变长
  - 软件结构混乱
  - 后续扩展困难
  - 数据连续输入时更容易丢包或处理不及时
- 更好的结构是：**回调搬运数据，主循环处理业务。**

设计思想

这是一种非常典型的嵌入式软件设计思想：

中断/回调收集数据，前台处理复杂逻辑。

共享数据与同步问题

- 接收缓冲区、接收长度和接收标志位通常同时被**回调函数**与**主循环**访问。
- 这类变量通常需要考虑：
  - 使用 `volatile`，防止编译器错误优化
  - 拷贝共享缓冲区时进入临界区，避免读写冲突
  - 控制缓冲区长度，防止越界
- 如果忽略这些同步问题，程序可能表现为：
  - 偶发解析错误
  - 字符串截断
  - 标志位状态异常

目录

1 背景与定位

2 USB 通信基础

3 CDC 虚拟串口原理

4 STM32 中的 USB CDC

5 程序设计要点与流程图

程序设计的总体思路

1. 先完成系统时钟、GPIO、USB Device 和 CDC 中间件初始化。
2. 等待 USB 枚举稳定，再发送欢迎信息或启动提示。
3. 使用“**接收缓冲区 + 长度变量 + 标志位**”构成应用层接收通道。
4. 在 `CDC_Receive_FS()` 中只做最小动作：复制数据、记录长度、置位标志、重新开启接收。
5. 主循环检测标志位后再解析命令、组织响应并调用发送接口。

STM32 USB 虚拟串口程序设计要点

- **初始化顺序要清晰**：时钟错误会直接导致设备无法枚举。
- **发送函数要可恢复**：应考虑 USBD_BUSY、超时和重试策略。
- **接收路径要简单**：优先把数据从底层搬到自己的应用缓冲区。
- **命令处理要放在主循环**：便于调试、扩展和维护。
- **字符串处理要谨慎**：注意 \r/\n、结尾 \0、缓冲区上限。
- **程序要有可观测性**：欢迎信息、状态命令、心跳输出都能显著降低调试难度。

典型模块划分

main.c

- 系统初始化
- 主循环
- 周期任务
- 调用命令处理函数

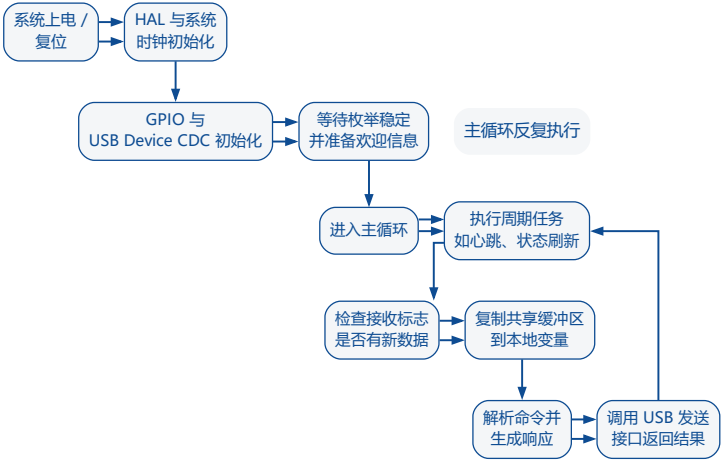
usbd_cdc_if.c

- 发送接口封装
- 接收回调
- 应用共享缓冲区
- 与 USB Device 栈对接

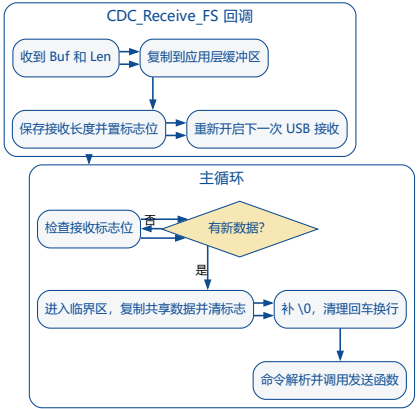
模块边界

协议栈负责 USB 通信框架，应用层负责“收到什么、如何解释、返回什么”。

程序总体流程图


系统上电 / 复位
HAL 与系统时钟初始化
GPIO 与 USB Device CDC 初始化
等待枚举稳定并准备欢迎信息
主循环反复执行
进入主循环
执行周期任务如心跳、状态刷新
检查接收标志是否有新数据
复制共享缓冲区到本地变量
解析命令并生成响应
调用 USB 发送接口返回结果

接收回调与主循环协作流程图



阅读要点

- 回调函数负责收包、搬运数据和置位标志
- 主循环负责读取共享数据、清标志并处理业务
- 虚线表示通过共享缓冲区和标志位完成前后台协作

与后续实验的衔接

- 后续实验部分并不是重新设计一套程序，而是在本节给出的结构上填入具体代码。
- CubeMX 负责生成 **USB Device CDC 基础框架**，学生主要补充的是：
  - 发送封装函数
  - 接收回调中的应用层搬运逻辑
  - 主循环中的命令解析与响应生成
- 本节两张流程图可以直接作为实验报告中的**程序设计说明**和**程序流程图**基础。

总结

- USB 虚拟串口本质上是**USB CDC 设备**，不是底层仍然使用 UART 的“假串口”。
- 电脑侧出现的 COM 口来自**驱动抽象**，其前提是枚举、描述符和类匹配都正确。
- 对 STM32 而言，理解**48 MHz 时钟**、**发送忙状态**、**接收回调**和**主循环协作**是掌握该主题的关键。
- 在工程实现上，推荐采用“**回调收数据，主循环做业务**”的结构，这也是后续实验的标准基础框架。